

Practice Exam (Unofficial) - Solutions

April 6, 2025

Part 1: Conceptual Questions - Solutions

1. The command is `ls -a` to list including hidden files. Or `ls -la` (`= ls -l -a`) if you want detailed list information.

2.

```
git add feature.c
git commit -m "Implement new feature"
git push
```

3. **Result:** 2.000000.

Why: `5 / 2` performs integer division, resulting in 2. This integer is then converted to a float 2.0 for assignment.

Fix: Ensure floating-point division occurs by making at least one operand a float.

Examples:

```
float result = 5.0 / 2;
// or
float result = 5 / 2.0;
// or
float result = (float)5 / 2;
```

4. The type of `ptr` is pointer to char.
5. It is pass-by-reference. The address of `num` is passed by value. The function receives a copy of the address (a pointer). By dereferencing the pointer (`*x`), the function accesses and modifies the original `num`.
Output: `num: 15`.
6. The null terminator character, `\0`, signifies the end. `strlen(s)` does **not** include the null terminator in its count.
7. $15 \pmod{11} = 4$
 $25 \pmod{11} = 3$
 $35 \pmod{11} = 2$
The keys map to indices 4, 3, and 2 respectively.

8. Problem: Buffer overflow. `strcpy` does not check destination size and will write past the end of `dest` if `src` (including its null terminator) is longer than 9 characters. Safer alternative: `strncpy`. Use it carefully to ensure null termination:

```
strncpy(dest, src, sizeof(dest) - 1);  
dest[sizeof(dest) - 1] = '\\0';
```

9. **Global Variable:** Declared outside functions. Scope: Accessible from point of declaration to end of file (and potentially other files via `extern`). Lifetime: Entire program execution.

Local Variable: Declared inside a function/block. Scope: Limited to the function/block where declared. Lifetime: Exists only while the function/block is executing (unless `static`).

10. Effect: Changes the variable's lifetime from automatic (recreated/destroyed each call) to static (exists for program duration, initialized only once).

What happens to `count`: It retains its value between calls to `counter()`. It's initialized to 0 only before the first call, then increments to 1, 2, 3, ... on subsequent calls.

11. Compile command:

```
gcc main.c utils.c -o myprogram
```

Run command:

```
./myprogram
```

12. There are two errors:

- (a) Accessing member `id` via pointer: `recPtr.id` should use the arrow operator `->` because `recPtr` is a pointer.

Fix: `recPtr->id = 101;`

- (b) Assigning string literal to char array: `recPtr.name = "record1";` is incorrect. Arrays cannot be assigned directly. Use `strcpy` or `strncpy`.

Fix: `strcpy(recPtr->name, "record1");` Note the use of `->` here as well.

13. `malloc(size)` allocates a single block of `size` bytes; the memory content is uninitialized (garbage values, whatever was there on the heap before). `calloc(n, size_elem)` allocates memory for an array of `n` elements, each of `size_elem` bytes; the allocated memory is initialized to all zero bytes.

14. A node in a singly linked list contains data and a pointer to the *next* node only. A node in a doubly linked list contains data, a pointer to the *next* node, and a pointer to the *previous* node, allowing traversal in both directions.

15. `b calculate_sum` (or `break calculate_sum`).

16. The macro expands directly to $5 + 2 * 10$. Due to operator precedence ($*$ before $+$), this evaluates as $5 + (2 * 10) = 5 + 20 = 25$.

This doesn't match the expected result of 70. The macro arguments are not evaluated before substitution, and operator precedence applies to the expanded code.

Fix: Add parentheses around the arguments and the entire expression in the macro definition:

```
#define SCALE(x, factor) ((x) * (factor))
```

17. `arr` is an array that contains 20 elements. Each element is a pointer to a double. The size is $20 * \text{sizeof}(\text{double} *)$, which is 160 bytes.

18. Trace:

- Outer loop $i = 1$: prints "hi",
Inner loop $j = 0$ ($0 < 1$): prints "hi".

Total for $i=1$: 2.

- Outer loop $i = 2$: prints "hi",
Inner loop $j = 0$ ($0 < 2$): prints "hi",
Inner loop $j = 1$ ($1 < 2$): prints "hi".

Total for $i=2$: 3.

Total: $2 + 3 = 5$ lines of "hi".

19. `data_buffer = (char *) realloc(data_buffer, new_size);`

Note: In real code, the result of `realloc` should be stored in a temp pointer first to check for failure before assigning back to `data_buffer`, to avoid losing the original pointer if `realloc` fails.

20. Catastrophic cancellation is the significant loss of precision that occurs when subtracting two floating-point numbers that are very close to each other in value. The leading, most significant digits cancel out, leaving fewer significant digits in the result, while the potential absolute error remains similar, leading to a large relative error. It's most likely to occur during the subtraction of nearly equal quantities.

Part 2: Data Structure Implementation - Solutions

a)

Advantage (Linked List): Dynamic Size: The stack can grow or shrink as needed at runtime, limited only by available memory. Array-based stacks have a fixed size determined at compile time or allocation time, which can lead to overflow if exceeded.

Disadvantage (Linked List): Inefficient Search: Nodes can be scattered in memory. We rely on the pointer from previous node to tell us where in memory to go for the next node. So, we cannot return the middle of the list in $O(1)$ like we can with the middle element of a

fixed-size array (which has a contiguous block of memory, so it can do fast pointer arithmetic to find the address).

b)

```
// Node structure for the linked list
typedef struct StackNode {
    int data;
    struct StackNode *next;
} StackNode;

// Stack structure containing a pointer to the top node
typedef struct Stack {
    StackNode *top;
} Stack;
```

c)

```
Stack* createStack() {
    // Allocate memory for the Stack structure itself
    Stack *s = (Stack*)malloc(sizeof(Stack));
    // Check if allocation succeeded
    if (s == NULL) {
        return NULL; // Return NULL on failure
    }
    // Initialize the top pointer to NULL, signifying an empty stack
    s->top = NULL;
    return s; // Return the pointer to the newly created stack
}
```

d)

```
// Function takes Stack* and int data
int push(Stack *s, int data) {
    if (s == NULL) {
        printf("Error: invalid stack pointer\n");
        return -1;
    }

    // Allocate memory for the new node
    StackNode *newNode = (StackNode*)malloc(sizeof(StackNode));
    if (newNode == NULL) {
        // Memory allocation failed
        return -1;
    }
}
```

```

}

// Set the data for the new node
newNode->data = data;
// Link the new node to the current top of the stack
newNode->next = s->top;
// Update the stack's top pointer to point to the new node
s->top = newNode;

return 1; // Success
}

```

e)

```

// Function takes Stack* to be popped from
int pop(Stack *s) {
    // Check for invalid stack pointer or empty stack
    if (s == NULL || s->top == NULL) {
        printf("Error: stack invalid or underflow\n");
        return -1;
    }

    // Store pointer to the current top node
    StackNode *tempNode = s->top;
    // Get the data from the top node
    int poppedData = tempNode->data;

    // Update the stack's top pointer to the next node in the list
    s->top = s->top->next; // or s->top = tempNode->next;

    // Free the memory of the removed node
    free(tempNode);

    // Return the popped data
    return poppedData;
}

```

f)

```

// Function takes Stack* to be freed
void freeStack(Stack *s) {
    // Check if the stack pointer is valid
    if (s == NULL) {
        return;
    }
}

```

```

StackNode *currentNode = s->top;
StackNode *nextNode;

// Iterate through the linked list of nodes
while (currentNode != NULL) {
    nextNode = currentNode->next; // Save pointer to the next
    node
    free(currentNode);           // Free the current node
    currentNode = nextNode;      // Move to the next node
}

// After freeing all nodes, free the Stack itself
free(s);
}

```